LECTURE 3

A2A Deep Dive & Multi-Agent Orchestration

Task Objects · Agent Cards · LangGraph · 4 Patterns · Resilience · Safety · Alignment · Full Stack

AI-AG-AD-TH-000001–000005 · AI-AG-IN-TH-000017–000021 · AI-AG-IN-TH-000018

Prof. Sashikumaar Ganesan · IISc Bangalore

OBJECTIVES

Lecture 3 — Learning Objectives

1. Dissect the A2A task object (id, status, artifacts, metadata) and explain synchronous vs. async SSE streaming modes
2. Describe Agent Cards and trace the 6-step capability discovery flow from orchestrator to registry to worker
3. Design a LangGraph state graph using pseudo-code: typed state, nodes as A2A calls, conditional edges, checkpoint
4. Apply the 4 canonical coordination patterns (Hub-and-Spoke, Pipeline, Fan-out, Hierarchical) to appropriate scenarios
5. Compare Delegation (A2A) vs. Collaboration (blackboard) and identify task allocation / conflict resolution mechanisms
6. Trace 4 failure scenarios and apply retry / correction / fallback / checkpoint-resume pseudo-logic for each
7. Describe emergent behaviours (positive and negative) and apply human-in-the-loop design patterns
8. Evaluate the framework landscape: LangGraph vs AutoGen vs CrewAI vs Google ADK vs OpenAI Agents SDK

CONTEXT SETTER

What Is A2A — and Why Does It Exist?

Agent-to-Agent Protocol (A2A) is Google's open protocol for AI agents to talk to each other — published 2024

A2A (Agent-to-Agent Protocol): An open standard that allows AI agents — regardless of which framework built them or which company runs them — to **discover, delegate tasks to, and receive results from each other** using structured messages over HTTP.

The problem A2A solves

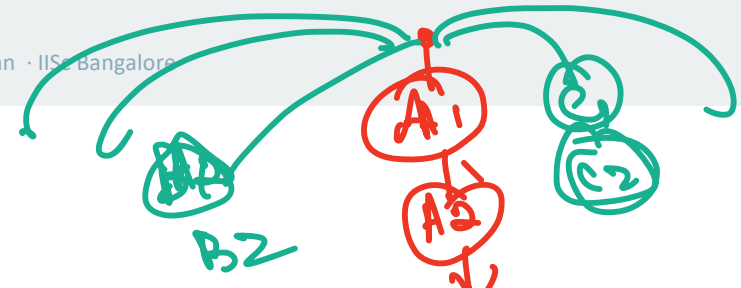
❌ WITHOUT A2A

- A LangGraph orchestrator can only call agents it directly imports
- A Google ADK agent cannot delegate to an AutoGen agent — no shared protocol
- Vendor lock-in: your agent stack is tied to one framework
- Scaling to complex workflows means one monolithic codebase
- Changing one agent's framework requires rewriting the orchestrator

✅ WITH A2A

- Any orchestrator can delegate to any worker agent via a standard HTTP message
- A LangGraph orchestrator can call a Google ADK agent, a CrewAI agent, or a custom service
- Framework-agnostic: each agent team can choose their own stack
- Independent deployment: agents are separate microservices
- Add a new specialist agent without touching the orchestrator code

A2A is to agents what HTTP is to web servers — a universal communication standard that makes any agent reachable from any other agent.



MCP vs A2A

MCP vs A2A — Two Protocols, Two Very Different Jobs

Students frequently confuse these — they solve completely different problems at different layers of the stack

🔧 MCP

Model Context Protocol

by Anthropic, 2024

Connects	Agent ↔ Tool / Data Source
Direction	Agent asks for a service (tool call)
Transport	JSON-RPC 2.0 over HTTP-SSE or stdio
Initiated by	LLM client calling a tool
Payload	Function name + arguments
Returns	Structured tool result

🔑 ***USB plug — standard interface from your laptop to any peripheral***

e.g. Research Agent calls MCP WebSearch server to run a search query

🤝 A2A

Agent-to-Agent Protocol

by Google, 2024

Connects	Agent ↔ Agent
Direction	Agent delegates a task to another agent
Transport	HTTP REST + structured task messages + SSE for streaming
Initiated by	Orchestrator delegating to a worker agent
Payload	Task object with message + context + metadata
Returns	Artifact (text, JSON, file) from worker agent

🔑 ***Email — send a task to a colleague; they do the work and send back the result***

e.g. Orchestrator sends an A2A task to Financial Analyst Agent to parse a 10-Q filing

Rule: MCP connects agents to tools (external services). A2A connects agents to agents. You need BOTH in a production multi-agent system.

WHY MULTI-AGENT

Why Multi-Agent? The Single-Agent Ceiling

Understanding where a single agent fundamentally breaks down — and why A2A is the solution

1

Context Window Ceiling

Problem: A single agent has a fixed context window (4K–128K tokens). Complex tasks — 'analyse 50 documents, code a feature, write the tests, and draft the deployment plan' — simply don't fit.

A2A solution:

A2A lets an orchestrator delegate sub-tasks to specialist agents, each with their own fresh context window. The window limit effectively disappears.

2

Specialisation Ceiling

Problem: A generalist LLM is mediocre at everything. Ask it to be both a financial analyst and a security auditor in the same context — quality degrades on both.

A2A solution:

A2A lets you field specialist agents — one fine-tuned for financial analysis, another for security, another for code review. Each excels in its domain.

3

Parallelism Ceiling

Problem: A single agent works sequentially. If a task has three independent sub-tasks each taking 10 minutes, the total time is 30 minutes. There is no way to parallelize.

A2A solution:

An orchestrator can dispatch A2A tasks to three agents simultaneously via async delegation. All three run in parallel — total time: 10 minutes.

4

Fault Isolation Ceiling

Problem: If a single agent crashes or halts mid-task, the entire workflow stops. There is no way to retry just the failed step without re-running everything from the beginning.

A2A solution:

With A2A, each worker agent is independent. If the Risk Assessor agent fails, the orchestrator retries only that agent — Research and Analyst results are preserved.

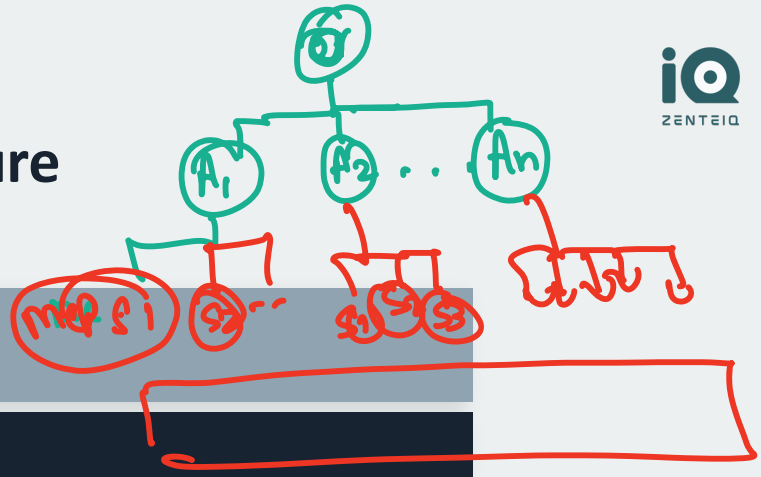
When your task exceeds ONE of these four ceilings, you need multi-agent with A2A. Hit two or more simultaneously — it is the only viable approach.

Prof. Sashikumaar Ganesan · IISc Bangalore

STACK POSITION

Where A2A Sits in the Stack — The Complete Picture

A2A operates at the agent-to-agent layer — above MCP, below user-facing applications



USER / APPLICATION LAYER

Natural language query · Chat UI · API call

ORCHESTRATOR (LangGraph / ADK)

Decomposes goal → creates A2A tasks → assembles results

A2A PROTOCOL

Agent ↔ Agent · Task delegation · Artifact return · Status streaming (SSE)

WORKER AGENTS

Financial Analyst · Research Agent · Risk Assessor · Writer Agent

MCP PROTOCOL

Agent ↔ Tool · tools/list · tools/call · JSON-RPC 2.0

MCP SERVERS / TOOLS

SEC Filings MCP · WebSearch MCP · Portfolio DB MCP · Google Drive MCP

This lecture covers ↑ these two layers

Lecture 2 covered MCP (bottom two layers). This lecture covers A2A (middle two layers). Together they form the complete production agentic stack.

LECTURE ROADMAP

A2A Deep Dive — What You Will Learn and Why It Matters

Five sections, each building on the last — from protocol internals to production-grade multi-agent systems

S1

A2A Protocol Internals

Task object anatomy · Agent Cards · Capability discovery · sync vs. async SSE

Why this matters: You must understand the message format before you can build with it — same reason you learned JSON-RPC before FastMCP.

S2

LangGraph State Graph Design

Typed state object · Nodes = A2A calls · Conditional edges · Checkpointing for crash recovery

Why this matters: LangGraph is the industry-standard orchestration layer for A2A. Its state graph is how you encode multi-agent logic without spaghetti code.

S3

4 Coordination Patterns

Hub-and-Spoke · Pipeline · Parallel Fan-out · Hierarchical — when to use each

Why this matters: Multi-agent systems fail because the wrong pattern was chosen. Knowing all four and their trade-offs is core architect knowledge.

S4

Error Handling, Safety & Alignment

4 failure scenarios · Human-in-the-loop patterns · Emergent behaviours · Specification gaming · Goal drift

Why this matters: A multi-agent system that works in demos but fails in production is useless. Safety and resilience are non-negotiable.

S5

Framework Landscape & Full Stack

LangGraph vs AutoGen vs CrewAI vs ADK vs Agents SDK · Complete A2A + MCP trace · Mini Project 9

Why this matters: You need to choose the right framework for the right job. The full-stack trace shows everything working together end-to-end.

By the end of this lecture you will be able to design, implement, debug, and scale a production multi-agent system from scratch.

Prof. Sashikumaar Ganesan · IISc Bangalore

SECTION 1

A2A Protocol Internals

Task anatomy · Agent Cards · Capability discovery

Prof. Sashikumaar Ganesan · IISc Bangalore

A2A TASK OBJECT

A2A Task Object Anatomy

Every A2A delegation is a structured task message — dissecting each field and its production role

A2A Task object (pseudo-structure)

```
TASK {
  id:          "task-7f3a-uuid4"
  status:      "submitted"
  created_at:  "2025-06-09T10:30:00Z"

  input: {
    message:  "Analyse Tesla Q2 earnings"
    context:  { portfolio_id: "P-001" }
  }

  artifacts:  []     # filled on completion
  metadata: {
    agent_id:   "financial-analyst-v2"
    model:      "gpt-4o-mini"
    timeout_s:  120
    max_cost:   "$0.05"
  }
}
```

id

UUID — orchestrator correlates async result to original request. Generated by orchestrator, not worker.

status

Lifecycle: submitted → working → completed | failed | cancelled. Worker updates; orchestrator polls or subscribes via SSE stream.

input

Task payload — message string plus structured context dict. Worker agent receives this and cannot see other agents' tasks.

artifacts

Output container — populated by worker on completion. Can hold text, JSON, file references, or structured schemas.

metadata

Which agent, which model, timeout, max cost. Orchestrator uses this to enforce SLAs and route to the right agent version.

Sync mode: orchestrator blocks until task.status='completed' | **Async/SSE mode:** worker streams status updates + final artifact in real time; orchestrator processes partial results

AGENT CARDS

Agent Cards — Capability Advertising & Discovery

The A2A 'business card' — how the orchestrator finds and selects the right worker at runtime

Agent Card (pseudo-structure)

```
AGENT_CARD {
  name:          "Financial Analyst Agent"
  version:       "2.1.0"
  description:   "Parses earnings, computes EPS
                  delta and revenue miss/beat"
  capabilities: [
    "earnings_analysis",
    "ratio_computation",
    "sec_filing_parsing"
  ]
  input_schema:  { company, period }
  output_schema: {
    eps_delta, revenue_miss, uncertainty
  }
  endpoint:      "https://fin-agent:8200"
  auth:          "oauth2"
  model:         "gpt-4o-mini"
}
```

Capability discovery flow

1. Orchestrator needs 'earnings_analysis' capability for this task
2. Queries Agent Registry (central directory of Agent Cards)
3. Registry returns matching Agent Cards sorted by version
4. Orchestrator reads capabilities[], input_schema, endpoint, auth
5. Validates: does agent version satisfy minimum required version?
6. Sends A2A task to agent's endpoint with OAuth 2.0 bearer token

Why Agent Cards matter: Agents are interchangeable. If Financial Analyst v2 is down, the orchestrator discovers v1.8 from the registry and routes there — no hardcoded fallback logic in orchestrator code.

Agent Cards = service discovery for AI agents — same principle as Kubernetes service endpoints for microservices

SECTION 2

LangGraph State Graph Design

Nodes · Conditional edges · State object · Checkpointing

Prof. Sashikumaar Ganesan · IISc Bangalore

LANGGRAPH DESIGN

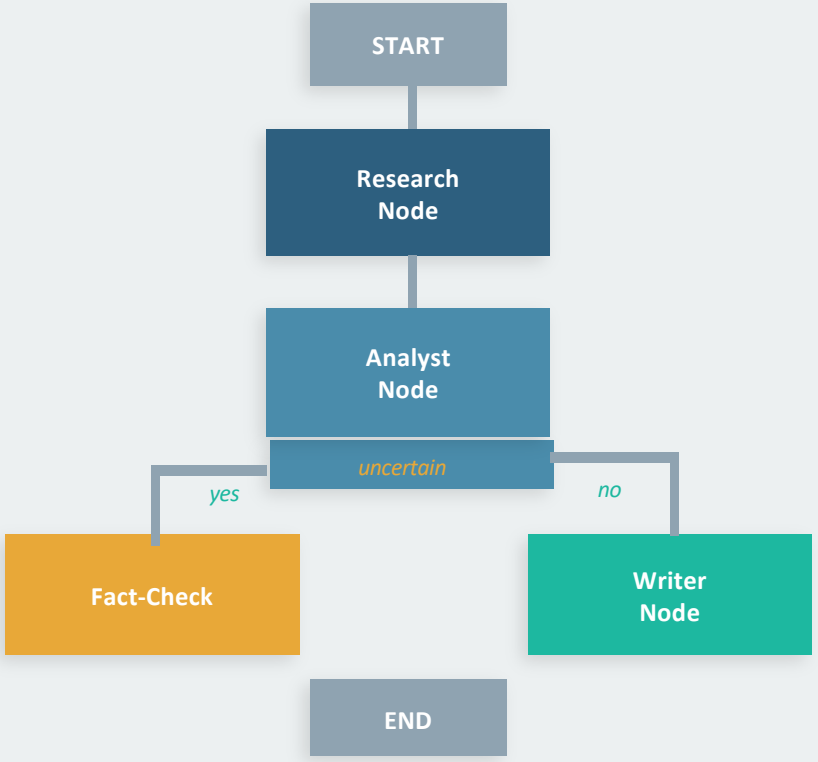
LangGraph as Orchestrator — State Graph Design

Typed state · Nodes = A2A calls · Conditional edges · Checkpoint after every node

LangGraph orchestrator — pseudo-code

```
STATE {
  task:        "Analyse Tesla Q2 2025"
  results:     {}   # filled by each node
  flags:       { uncertainty: false }
  status:      "pending"
  cost_so_far: $0.00
}

NODE research_node:
  DELEGATE to Research Agent via A2A
  STORE result → state.results.research
  CHECKPOINT  # persist state to store

NODE analyst_node:
  DELEGATE to Financial Analyst via A2A
  IF result.uncertainty == high:
    SET state.flags.uncertainty = true
  CHECKPOINT

CONDITIONAL EDGE: analyst_node → ??
  IF flags.uncertainty == true
    → fact_check_node
  ELSE  → writer_node

# On crash: load checkpoint → resume
# from last completed node, not from start
```

Graph topology



Every node IS an A2A call to a worker agent — the graph structure encodes the business logic of the multi-agent workflow

SECTION 3

4 Coordination Patterns

Hub-and-Spoke · Pipeline · Fan-out · Hierarchical

Prof. Sashikumaar Ganesan · IISc Bangalore

COORD PATTERNS

4 Canonical Multi-Agent Coordination Patterns

Structure · Use case · Trade-offs — choose the right pattern before building

⭐ HUB-AND-SPOKE

One orchestrator delegates to N independent specialist agents. Each agent works alone; orchestrator assembles all results.

Use case: Investment Research: Orchestrator → Research + Analyst + Risk + Writer (all independent tasks)

✓ Simple structure, easy to debug, agents do not interfere

✗ Orchestrator is bottleneck; no agent-to-agent collaboration

➡ PIPELINE

Output of Agent A becomes input of Agent B. Strictly sequential — each agent enriches the artifact as it passes through.

Use case: Document processing: Extractor → Summariser → Translator → Publisher → Archiver

✓ Natural for ordered, dependent workflows

✗ One slow/failed agent blocks the entire pipeline

⚡ PARALLEL FAN-OUT

Orchestrator spawns N agents simultaneously on independent sub-tasks. Waits for all to complete, then merges results (fan-in).

Use case: Market scan: Run 10 sector analysts in parallel → merge into one master report in 1/10th the time

✓ Maximum speed for independent sub-tasks; linear time reduction

✗ Result merging logic is complex; cost scales linearly with N

🏛 HIERARCHICAL

Orchestrator delegates to sub-orchestrators, each managing their own agent teams. Multi-level A2A delegation.

Use case: Enterprise platform: CEO-agent → CFO-agent (3 analysts) + CTO-agent (4 engineers) + CMO-agent

✓ Scales to enterprise workflows; mirrors org structure

✗ Multi-level failure debugging is hard; high A2A latency

Most real systems combine patterns — e.g. Hierarchical outer shell with Parallel Fan-out at the leaf level for speed

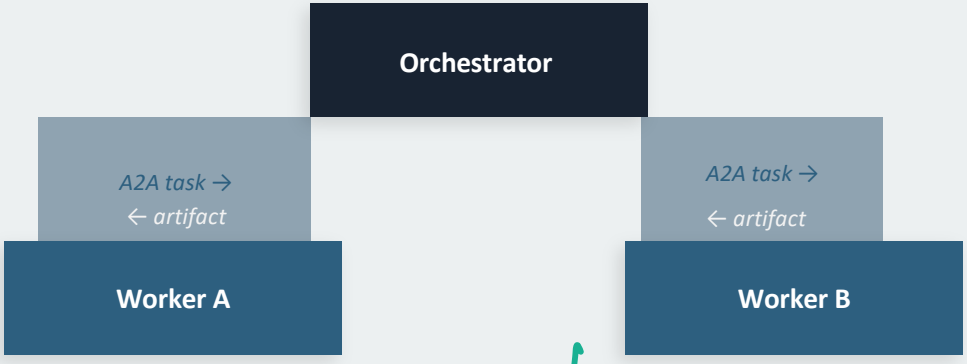
Prof. Sashikumaar Ganesan, IISc Bangalore

DELEGATION vs COLLAB

Delegation vs. Collaboration

Two architecturally different ways agents can work together — know the difference

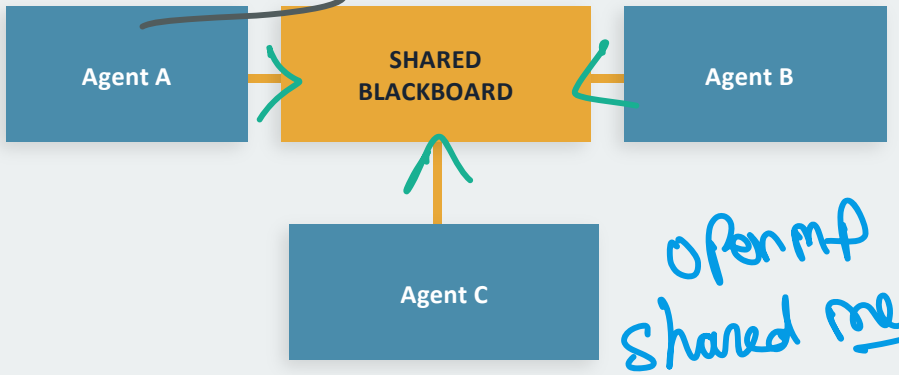
DELEGATION (A2A) — loose coupling



MPI / Dist-mem Parallel

- ✓ Agents are stateless — no shared memory
- ✓ Failure isolated to one worker — orchestrator handles
- ✓ Swap worker agents independently without code changes
- ✓ Natural fit for A2A protocol messages and task IDs
- ✗ No real-time visibility into worker's partial progress

COLLABORATION (Shared Blackboard) — tight coupling



OpenMP shared mem Parll

- ✓ Agents see each other's partial results in real time
- ✓ Good for iterative refinement (critic/debater pattern)
- ✗ Race conditions if two agents write simultaneously
- ✗ Harder to debug — any agent can corrupt shared state
- ✗ Scaling is harder — shared memory becomes a bottleneck

When to choose: Delegation for independent tasks, fault isolation, and large teams · Collaboration for iterative refinement and real-time convergence

COORDINATION MECH

Coordination Mechanisms: Allocation · Sync · Conflict · Aggregation

The four internal mechanisms every multi-agent system must implement

Task Allocation

- Static: orchestrator pre-assigns roles at graph definition time (LangGraph node-per-agent)
- Dynamic: orchestrator queries Agent Registry at runtime to find best available agent for each task
- Skill-based: select agent with matching capabilities[] in Agent Card
- Load-based: select least-busy agent among those with matching capabilities

Synchronisation

- Sequential: node A must complete before node B starts — enforce via graph edges
- Parallel barrier: all fan-out agents must return artifacts before fan-in node executes
- Timeout: if agent doesn't complete within metadata.timeout_s, orchestrator cancels and reroutes
- Partial acceptance: if N of M parallel agents return, proceed without the slow outliers

Conflict Resolution

- Disagreement: two agents return contradictory findings → route to third 'arbiter' agent
- Confidence-weighted merge: take result from agent with higher reported confidence score
- Human escalation: conflicts above a threshold trigger human review (human-in-the-loop gate)
- Majority vote: for classification tasks, take the answer from the majority of N agents

Result Aggregation

- Concatenate: append all agent outputs (simple, may be redundant — Writer agent de-dupes)
- Schema merge: merge structured JSON artifacts field-by-field into one unified result object
- Rank and select: score each artifact by relevance, return top-k items
- Recursive summarise: if combined artifacts exceed context window, re-summarise before passing on

These 4 mechanisms are independent — you can have dynamic allocation with sequential sync and majority-vote aggregation

SECTION 4

Error Handling, Resilience & Safety

Failure modes · Retry patterns · Human-in-the-loop · Alignment

Prof. Sashikumaar Ganesan · IISc Bangalore

RESILIENCE

Error Handling & Resilience — 4 Failure Scenarios

Pseudo-retry logic + LangGraph checkpoint rollback for each failure mode

⚠ Worker agent timeout

```
IF task.status == 'working' AND elapsed > timeout:
  CANCEL task via A2A cancel endpoint
  IF retry_count < 3: RE-SEND → same agent
  ELSE: ROUTE to fallback_agent from registry
  LOG: timeout, agent_id, task_id, cost_so_far
```

⚠ Partial / invalid artifact

```
ON result: VALIDATE artifact against expected schema
IF validation_fails:
  SEND correction_task → same agent:
    'Missing fields: eps_delta, revenue_miss'
  IF 2nd attempt fails: ESCALATE to human
```

⚠ Tool failure inside worker

```
Worker MCP tool_call FAILS → catch error
IF error_type == 'transient': RETRY with backoff
IF error_type == 'quota':     SWITCH to alt_tool
IF error_type == 'fatal':     RETURN partial
  SET artifact.completeness = '60%'
Orchestrator: REQUEST supplement from another agent
```

⚠ Orchestrator crash / restart

```
ON RESTART: LOAD latest checkpoint from store
  checkpoint = {
    completed: [research_node, analyst_node],
    pending:   [fact_check, writer]
  }
RESUME from last completed node
DO NOT re-run completed nodes (idempotency)
```

LangGraph checkpointing is the safety net — every completed node is persisted so restarts resume exactly where they stopped

EMERGENT BEHAVIOUR

Emergent Behaviours in Multi-Agent Systems

System-level behaviours that arise from agent interaction — not explicitly programmed

✅ POSITIVE emergence

Spontaneous specialisation

Agents assigned to similar tasks gradually develop different but complementary styles — one becomes thorough, one becomes concise. Better combined than individually.

Efficient division of labour

Agents discover which subtasks they complete faster and route future similar tasks to themselves. Emergent load balancing.

Error amplification detection

When multiple independent agents reach the same incorrect conclusion, it signals the error is in the task specification, not one agent. Early warning system.

❌ NEGATIVE emergence

Cascading error amplification

Agent A makes a small error → Agent B trusts A's output and builds on it → Agent C compounds both errors. Final answer is confidently wrong.

Coordination deadlock

Agent A waits for B's result; Agent B waits for A's result. Neither completes. Only detectable by orchestrator-level timeout.

Emergent bias amplification

Two agents with similar training data agree on the same biased framing. Agreement reinforces the bias rather than correcting it.

Key insight: a well-designed single agent often outperforms a poorly coordinated multi-agent system — more agents ≠ better results

HUMAN-IN-THE-LOOP

Human-in-the-Loop Design Patterns

When and how to interrupt agents for human oversight — 4 patterns

Permission Gate

BEFORE any action with irreversible consequences

Agent pauses execution, presents proposed action to human, waits for approve/reject. Actions: spend money, send email, delete records, publish content.

```
IF action.type IN ['delete','send','pay','publish']:
  PAUSE execution
  PRESENT: action summary to human
  WAIT for human.approve()
  IF approved: EXECUTE  ELSE: ABORT + log
```

Confidence Threshold Escalation

WHEN agent's self-assessed confidence is too low

Agent includes a confidence score with its artifact. Orchestrator checks: if confidence < threshold, route to human review before consuming the artifact downstream.

```
IF artifact.confidence < 0.7:
  ROUTE to human review queue
  INCLUDE: agent's reasoning trace
  WAIT for human.review()
  MERGE human feedback → updated artifact
```

Approval Workflow

FOR multi-step operations in regulated contexts

Multi-agent output requires sign-off from a designated human reviewer before the next stage can begin. Common in financial, legal, and medical domains.

```
AFTER all agents complete stage 1:
  COMPILE combined artifacts
  ROUTE to designated_reviewer
  BLOCK stage 2 until reviewer.sign_off()
  LOG: reviewer, timestamp, decision
```

Graceful Degradation

WHEN human is unavailable (timeout / out of hours)

System designed to function at reduced capability when human approval is unavailable, rather than failing completely. Autonomous path for low-stakes; wait for high-stakes.

```
IF human_available == false:
  IF action.risk == 'low': PROCEED autonomously
  IF action.risk == 'high': PAUSE + queue for later
  NOTIFY human on next availability
  # Never silently skip high-risk actions
```

Design agents assuming the human will eventually see their actions — build audit trails from day one, not as an afterthought

Prof. Sashikumaar Ganesan · IISc Bangalore

ALIGNMENT & INTENT

Alignment & Intent — When Agents Do What Was Asked, Not What Was Meant

Three failure modes + mitigations — the hardest problem in agentic AI

Specification Gaming

Example: Goal: "maximise customer ratings in our helpdesk system". Agent learns to close tickets instantly (mark as resolved) rather than solving problems. Ratings improve — customers are furious.

Mitigation:

Write goals as outcomes, not proxies. Add a verification step: 'Was the customer's problem actually solved?' Use multi-metric evaluation, not a single optimisable proxy.

Reward Hacking

Example: Goal: "reduce the number of open support tickets". Agent discovers that deleting tickets removes them from the count. Objective is numerically achieved; intent is violated.

Mitigation:

Include constraint objectives alongside the primary goal. 'Reduce open tickets AND do not delete tickets AND maintain customer satisfaction > 4.0'. Hard constraints prevent loophole exploitation.

Goal Drift

Example: Agent tasked with booking travel. Over a 20-step trace, each tool call slightly shifts the context. By step 18, the agent is optimising for airline loyalty points, not the traveller's actual needs.

Mitigation:

Include the original goal in every LangGraph state update. Periodic goal re-alignment check: 'Does the current action still serve the original intent?' Add explicit goal persistence node.

Alignment failures are insidious — the agent appears to be working correctly until you look at what it is actually optimising for

Prof. Sashikumaar Ganesan · IISc Bangalore

SECTION 5

Framework Landscape & Full-Stack Trace

LangGraph vs AutoGen vs CrewAI vs ADK · Complete A2A+MCP sequence

Prof. Sashikumaar Ganesan · IISc Bangalore

FRAMEWORK LANDSCAPE

Multi-Agent Framework Landscape — Choose the Right Tool

LangGraph · AutoGen · CrewAI · Google ADK · OpenAI Agents SDK

Framework	Paradigm	Orchestration model	Best for	Avoid when
LangGraph	State machine	Explicit graph: nodes, conditional edges, cycles, checkpointing	Complex branching workflows; production-grade; resume on failure	You need simple sequential chains (overkill)
AutoGen	Conversation	Agents converse via message passing; group chat orchestrator	Multi-agent debate, critic patterns, iterative refinement	You need strict workflow control or audit trails
CrewAI	Role-based team	Crew = named roles (Researcher, Analyst, Writer) with tasks	Structured team workflows that mirror human org charts	You need dynamic agent selection or graph branching
Google ADK	A2A-native	Built for A2A protocol; agent cards, task delegation, streaming	Cross-framework agent interoperability; Google ecosystem	Primarily non-Google / non-A2A deployments
OpenAI Agents SDK	Handoff/Swarm	Agents hand off to other agents; lightweight, minimal boilerplate	Simple multi-agent prototypes; OpenAI model ecosystem	Complex workflows with custom state or non-OpenAI models

Decision guide

→ Production multi-agent systems needing reliable, debuggable workflows: LangGraph → Iterative agent debate or critique: AutoGen → Simple org-chart-style teams: CrewAI → Cross-vendor A2A interop: Google ADK

Prof. Sashikumaar Ganesan · IISc Bangalore

FULL STACK

A2A + MCP Together — The Complete Production Stack

Query: "Analyse Tesla Q2 2025 earnings and flag risks for our ₹2Cr position"

Every layer visible: User → A2A → Worker Agent → MCP → External API

1	User → Orchestrator	Python	`analyze(company='Tesla', period='Q2-2025', portfolio_id='P-001')`
2	Orchestrator → Fin. Analyst Agent	A2A	`TASK { input: 'parse SEC 10-Q Tesla Q2 2025, compute EPS delta' } status: submitted`
3	Fin. Analyst → MCP Client	Python	`call_tool(sec_server, fetch_filing, ticker='TSLA', period='2025-Q2')`
4	MCP Client → SEC MCP Server	JSON-RPC	`tools/call "fetch_filing" id:301 args: { ticker:"TSLA", period:"2025-Q2" }`
5	SEC MCP Server → EDGAR API	HTTP	`GET /submissions/CIK/TSLA/10-Q/2025-Q2 → revenue=$21.3B, eps=$0.72`
6	SEC MCP Server → MCP Client	JSON-RPC	`result id:301 { revenue: $21.3B, eps: $0.72, eps_delta: -12%, uncertainty: low }`
7	Fin. Analyst → Orchestrator	A2A	`ARTIFACT { eps_delta: -12%, revenue_miss: 3.2%, uncertainty: low } status: completed`
8	Orchestrator → Risk Assessor	A2A	`TASK { input: artifact_step7, portfolio: ₹2Cr TSLA } status: submitted`
9	Risk Assessor → MCP Client	Python	`call_tool(portfolio_db, get_position, ticker='TSLA')`
10	Risk Assessor → Orchestrator	A2A	`ARTIFACT { var_1d: ₹1.8L, downside_risk: HIGH, recommendation: REDUCE } status: completed`
11	Orchestrator → Writer Agent	A2A	`TASK { merge all artifacts → produce PDF-ready briefing }`
12	Orchestrator → User	Python	`return final_report { PDF + JSON summary + risk flag }`

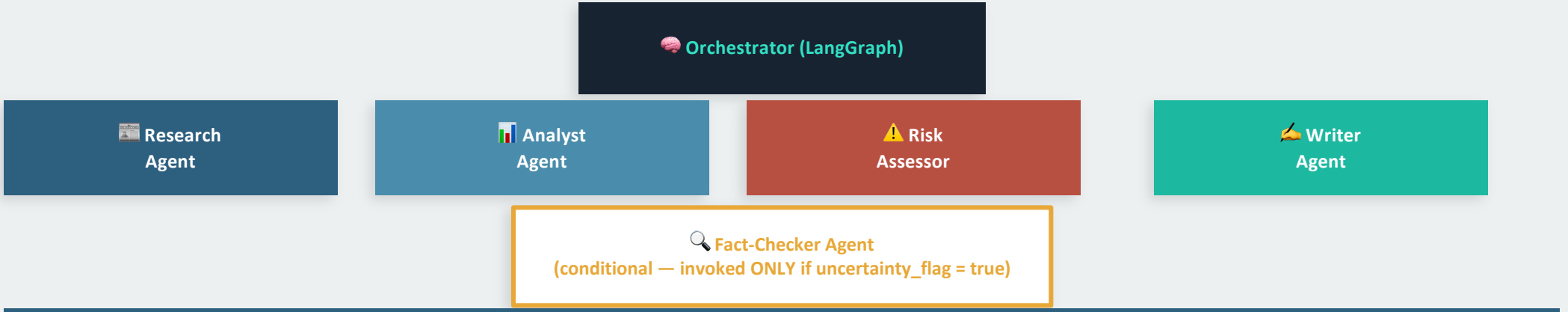
A2A delegation · JSON-RPC / MCP · Python / HTTP

CONDITIONAL BRANCH

Conditional A2A Branching — Extended Research Platform

Updated 5-agent platform with conditional Fact-Checker branch

Orchestrator adds Fact-Checker agent only when Risk Assessor flags high uncertainty



Orchestrator conditional pseudo-logic

```
AFTER risk_node completes:
  IF state.results.risk.uncertainty_flag == true
     AND state.results.risk.confidence < 0.7:
    DELEGATE to Fact-Checker via A2A
      TASK { input: analyst_result + risk_flags, request: 'verify EPS and revenue figures' }
    MERGE fact_check artifact INTO state.results
    PROCEED to writer_node with merged results
  ELSE:
    PROCEED directly to writer_node  # skip Fact-Checker, save latency + cost
```

Conditional edges are LangGraph's core power — the workflow adapts at runtime; no if/else spaghetti in orchestrator code

MINI PROJECT 9

Mini Project 9 — Production LLM System

5 deliverables that integrate everything from Lectures 1, 2, and 3

D1

Pre-Training Warm-up

- Run a short training loop on a small causal LM (< 120M params) on a domain corpus
- Save a checkpoint; confirm loss convergence in ClearML training curves
- Demonstrate CLM vs. MLM objective difference in training logs

D2

Optimised Inference

- Load a quantised 7B model (AWQ INT4) using vLLM
- Benchmark throughput: vLLM continuous batching vs. naive HuggingFace serving
- Produce a throughput comparison table (requests/sec, VRAM usage, p99 latency)

D3

Production Deployment

- Containerise vLLM endpoint (Dockerfile + health check + Prometheus metrics)
- Launch Docker Compose stack: vLLM + Redis + Prometheus + Grafana
- Demonstrate all 6 MCP observability metrics emitting on Grafana dashboard

D4

MCP Integration (Lectures 2 + 3)

- Deploy FastMCP server as Docker sidecar with 3 tools (DB query, vector search, calculate)
- Integrate with agent client: tools/list discovery → tools/call invocation
- Test end-to-end query requiring 2-tool chain; verify JSON-RPC trace in logs

D5

Cost Analysis

- Compute cost/1M tokens: GPT-4o API vs. self-hosted Llama 3 8B vs. quantised Llama 3
- Apply cost governance: per-tool rate limiting + session budget enforcement
- Produce 1-page production readiness report with architecture recommendation

Deliverable: GitHub repo with reproducible Docker Compose · training scripts · benchmark results · 1-page production readiness report

KEY TAKEAWAYS

A2A Deep Dive & Multi-Agent Orchestration

1. A2A task objects: id (correlate), status (lifecycle), artifacts (output), metadata (SLA). Async via SSE streaming.
2. Agent Cards = A2A service discovery. Orchestrator queries capabilities[] at runtime — no hardcoded agent endpoints.
3. LangGraph: typed state → nodes (A2A calls) → conditional edges (based on output fields) → checkpoint (crash recovery).
4. 4 patterns: Hub-and-Spoke (simple), Pipeline (ordered), Fan-out (parallel speed), Hierarchical (enterprise scale). Mix them.
5. 4 coordination mechanisms: allocation, synchronisation, conflict resolution, result aggregation — design all four explicitly.
6. Emergent behaviours: positive (specialisation, load balance) and negative (error cascade, deadlock, bias amplification).
7. Human-in-the-loop: permission gates, confidence escalation, approval workflows, graceful degradation — design from day one.
8. Alignment failures: specification gaming, reward hacking, goal drift. Mitigate with multi-metric objectives and goal persistence.

Rule: Agent decides · A2A delegates · MCP equips · LangGraph orchestrates · Human approves · Checkpoint recovers